NATIONAL ECONOMICS UNIVERSITY
FACULTY OF MATHEMATICAL ECONOMICS

MID-TERM TEST REPORT

Firm Data Hub – Vietnamese Firm Financial Panel Dataset of Firms 2020 – 2024

Supervisor: Tran Hung
Students:

- Vu Anh Son - 11245930
- Ngo Tuan Bach - 11245846
- Nguyen Khanh Duy - 11245870
- Thai Thanh Binh - 11245848
- Nguyen Lam Huy - 11245879

Class: DSEB 66B
Academic Year: 2025–2026

I. Introduction

This project develops an end-to-end data engineering workflow for building a firm-year panel dataset of Vietnamese listed companies. Its main objective is to transform scattered and partially unstructured corporate information into a clean, consistent, and analysis-ready database that can support research and reporting. Rather than treating data collection as a single extraction task, the project is designed as a complete pipeline that covers acquisition, processing, validation, storage, and final export.

The project integrates multiple categories of firm-level information, including ownership structure, financial statement indicators, cash flow measures, market variables, innovation-related indicators, and basic firm metadata. In practical terms, the system aims to produce a consolidated dataset of key variables at the firm-year level, while also preserving information about data sources, loading versions, and later corrections. This is especially important because the source materials originate from different formats and may require both automated extraction and manual review before they can be used reliably.

To address this challenge, the project combines automated and collaborative data-processing methods. Financial reports in PDF format are collected and processed through an Airflow-based pipeline that uses Google Cloud services and Gemini-based extraction to convert report contents into structured CSV outputs. These intermediate results are synchronized to Google Sheets, where team members can review, complete, and correct values manually. The manually curated data are then merged into a master sheet, from which ETL scripts load the validated records into a MySQL database organized around dimension tables, fact tables, data snapshots, and audit logs.

Beyond data loading, the project also includes a quality-control layer and a final output layer. Rule-based QC scripts check for logical inconsistencies, invalid values, missing information, and accounting anomalies, while a quick-fix mechanism allows manual corrections to be applied and logged transparently. Finally, the latest validated records are consolidated through a database view and exported as a final panel dataset for downstream analysis. Overall, the project provides a practical and reproducible framework for converting financial disclosures into a structured database that supports both operational data management and analytical use.

II. Data Sources

1 System Data Sources Dictionary

- **BCTC_Audited:** Provides data from audited financial statements. This type of data is officially collected from listed companies or stock exchanges (Company/Exchange).
- **FiinPro_Ownership:** The primary source for detailed reports on shareholder structure and ownership ratios (Ownership report). This data belongs to the `ownership` group and is extracted from the financial data provider, FiinPro.
- **AnnualReport:** Includes annual reports and other disclosure documents. This source provides qualitative, text-based data (`text_report`) published directly by the companies.
- **Vietstock:** A portal providing financial data and ratios. Data from Vietstock is manually extracted and entered by the team (`manual`).
- **Combined_Sources:** Data that is manually collected and cross-referenced by the team from various sources and providers (VariousSources) to supplement any missing data points (`manual`).

2 Firm Selection

The panel dataset covers 20 Vietnamese listed firms selected from different sectors of the economy. For each firm, the reference information includes the stock ticker, the full company name, and the level-2 industry classification. This information serves as the identification and classification layer of the dataset, allowing each firm-year observation to be linked consistently across the database while also supporting sector-level comparison and analysis. Table 1 presents the list of firms included in the panel.

Table 1: List of firms in the panel dataset

Ticker	Company Name	Industry L2 Name
BHN	Hanoi Beer Alcohol and Beverage Joint Stock Corporation	Food & Beverage
BSR	Binh Son Refining and Petrochemical Joint Stock Company	Oil & Gas
DBC	Dabaco Group Joint Stock Company	Food & Beverage
DGC	Duc Giang Chemicals Group Joint Stock Company	Chemicals
DHG	DHG Pharmaceutical Joint Stock Company	Health Care
DPM	PetroVietnam Fertilizer and Chemicals Corporation	Chemicals
GEE	Gelex Electric Joint Stock Company	Industrial Goods & Services
GMD	Gemadept Corporation	Industrial Goods & Services
GVR	Vietnam Rubber Group - Joint Stock Company	Chemicals
NKG	Nam Kim Steel Joint Stock Company	Basic Resources
PC1	PC1 Group Joint Stock Company	Construction & Materials
PVC	PetroVietnam Chemical and Services Corporation	Oil & Gas
PVD	PetroVietnam Drilling and Well Service Corporation	Oil & Gas
PVS	PetroVietnam Technical Services Corporation	Oil & Gas
RAL	Rang Dong Light Source and Vacuum Flask Joint Stock Company	Personal & Household Goods
TMS	Transimex Corporation	Industrial Goods & Services
VCG	Vietnam Construction and Import-Export Joint Stock Corporation	Construction & Materials
VGC	Viglacera Corporation - JSC	Construction & Materials
VHC	Vinh Hoan Corporation	Food & Beverage
VSC	Vietnam Container Shipping Joint Stock Company	Industrial Goods & Services

III. LLM-based Data Extraction

1 Purpose

The `airflow` folder serves as the workflow orchestration layer of the project. Its main purpose is to automate the end-to-end pipeline that transforms raw financial report PDF files into structured datasets

that can be reviewed, consolidated, and used for further analysis. Instead of executing each processing step manually, the project uses Apache Airflow to coordinate task execution, manage dependencies, and ensure that the workflow runs in a repeatable and organized manner.

This module is important because the overall system contains multiple stages, including data ingestion, AI-based extraction, cloud storage operations, spreadsheet synchronization, manual review, and final consolidation. Airflow provides a centralized mechanism to schedule and monitor these stages, making the processing pipeline more reliable and scalable.

```
airflow/
├── config/
│   └── airflow.cfg
├── dags/
│   ├── ocr_pipeline_dag.py
│   └── manual_collect_merge_dag.py
├── include/
│   ├── collect_upload_gcs.py
│   ├── processing.py
│   ├── data_select.py
│   ├── gcs_to_sheets.py
│   ├── gop_du_lieu.py
│   ├── manual_collect.py
│   ├── master_merge.py
│   ├── missing_tasks.py
│   ├── split_manual_files.py
│   ├── sheet_utils.py
│   └── config.py
├── docker-compose.yaml
├── dockerfile
└── requirements.txt
```

Figure 1: High-level structure of the `airflow` folder.

2 Main Responsibilities

The `airflow` module is responsible for several core operations in the project. First, it retrieves raw PDF files containing financial reports from Google Cloud Storage. Next, it triggers the extraction process that reads the PDF content and converts relevant financial information into structured tabular data. The extracted results are then saved as CSV files and uploaded back to cloud storage.

After extraction, the module synchronizes the generated data into Google Sheets, which function as a collaborative workspace for review and correction. It also creates separate manual-review files for different team members or groups, allowing the extracted data to be checked and completed by humans. Once manual review is done, the workflow recollects the reviewed data, merges it with the originally extracted data and supplementary information, and produces a final master sheet. In addition, it generates a missing-data task list to indicate which fields are still incomplete and who should handle them.

3 Folder Structure

The structure of the `airflow` folder reflects its role as an orchestration and integration module. The `dags` subfolder contains the Airflow DAGs, which define the main workflows executed by the system.

These DAGs specify the sequence of tasks, their dependencies, and their scheduling behavior.

The `include` subfolder contains reusable helper functions that implement the actual business logic used by the DAGs. These helper modules handle operations such as reading files from Google Cloud Storage, extracting structured financial data from PDFs, writing and reading Google Sheets, aggregating manual-review outputs, merging datasets, and generating missing-data tasks.

The `config` subfolder stores Airflow runtime configuration. In addition, the folder contains deployment files such as `docker-compose.yaml`, `dockerfile`, and `requirements.txt`, which define the environment needed to run Airflow services and install the required Python packages.

4 Core Workflows

The workflow logic in this module is mainly organized into two Airflow DAGs.

The first DAG is the OCR pipeline, which performs the automated extraction stage. Its process begins by listing all PDF files stored in a source bucket on Google Cloud Storage. Each PDF is then processed through an AI-based extraction routine that reads the report and returns structured financial data. The extracted output is written as CSV files and uploaded to a destination bucket. These CSV files are then synchronized to Google Sheets, both as individual outputs and as a consolidated worksheet. Finally, the workflow splits the consolidated output into multiple manual-review spreadsheets based on predefined team groups.

The second DAG is the manual collection and merge pipeline. This DAG runs on a daily schedule and is responsible for the human-in-the-loop stage of the system. It reads the data entered or corrected in the manual-review sheets, aggregates them into a single worksheet, and merges them with OCR-derived data and any additional input sheets. The result is written into a final master worksheet that represents the most complete and validated version of the dataset. After that, the DAG generates a missing-task worksheet to track which variables are still absent and assign those tasks to the appropriate team members.

5 Processing Logic

Although the folder is named `airflow`, the module does not directly perform extraction by itself. Its main role is orchestration. The detailed processing logic is implemented in helper modules under `include`, while Airflow manages when and how these modules are executed.

The extraction stage is not limited to traditional OCR in the narrow sense of converting images into plain text. Instead, the pipeline uses a Gemini-based AI extraction approach to analyze PDF financial statements and return structured financial variables in a predefined format. Therefore, the workflow is more accurately described as an AI-based document extraction pipeline with manual review support. After extraction, the outputs are standardized into CSV and spreadsheet formats so they can be consolidated and checked efficiently.

This design allows the project to combine automation and human validation. Automated extraction accelerates the initial data collection process, while manual review improves data quality and addresses cases where the extracted output is incomplete or uncertain.

6 Integration with External Services

The `airflow` module integrates with several external services that are central to the project's operation. Google Cloud Storage is used as the main storage layer for both input PDF reports and extracted CSV outputs. It acts as the bridge between raw documents and processed data.

Google Sheets is used as the collaborative review and consolidation platform. Individual extraction results can be uploaded into Sheets, and consolidated sheets are created to support manual validation, merging, and final data management. This makes it easier for multiple users to work on the data without requiring direct interaction with the codebase.

The extraction step also integrates with Vertex AI and Gemini, which are used to interpret the PDF reports and generate structured financial outputs. This AI-based approach enables the system to move beyond simple text recognition and perform field-level extraction tailored to the project's financial data requirements.

IV. Database Design

1 Architecture

Based on the requirements provided in the project assignment, our team first analyzed the problem and identified the main entities, attributes, and relationships necessary for storing the financial data collected from the selected firms.

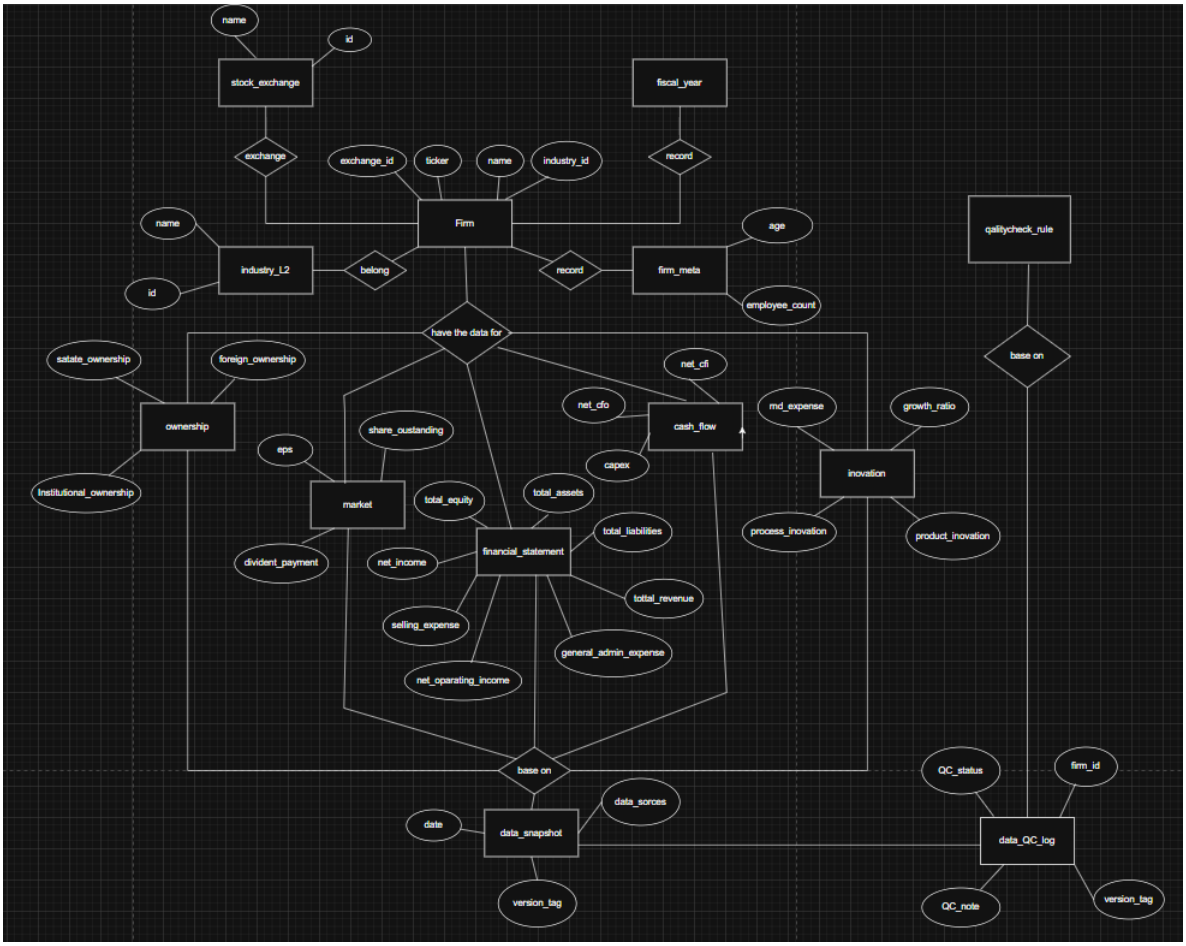

Figure 2: Entity Relationship Diagram

After mapping process, the database follows a DIM FACT SNAPSHOT architecture (Star schema) commonly used in analytical databases.

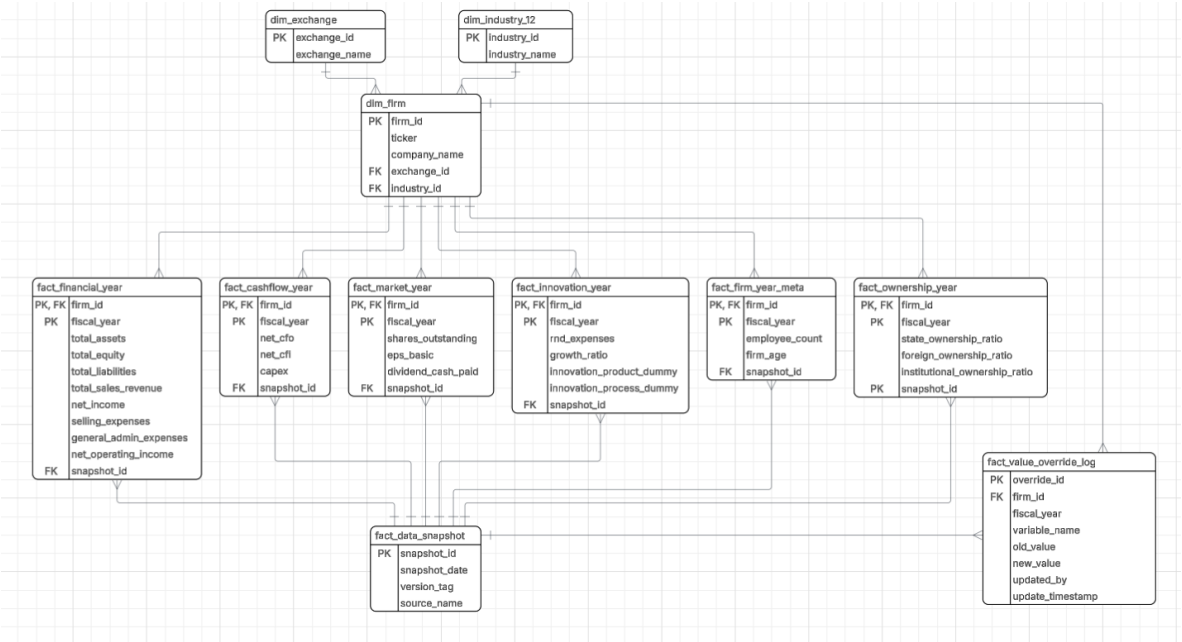

Figure 3: Relational Diagram

- Dimension tables store descriptive attributes.
- Fact tables store quantitative financial values.
- Snapshot tables track dataset versions.

This design allows flexible querying and supports panel data analysis.

2 Dimension Tables

The schema includes the following dimension tables:

Table 2: Dimension tables

Table	Key Attributes
`dim_firm`	`ticker` (PK), `company_name`, `exchange_id` (FK), `industry_id` (FK), `founded_year`
`dim_exchange`	`exchange_id` (PK), `exchange_code`, `exchange_name`, `country`
`dim_industry_l2`	`industry_id` (PK), `industry_name`, `sector`, `classification_standard`
`dim_data_source`	`source_id` (PK), `source_name`, `source_type`, `url`, `description`

The `dim_firm` table is the central anchor for all fact tables, with a unique constraint on ticker ensuring referential integrity across the schema.

3 Fact Tables

Financial measurements are distributed across specialized fact tables organized by data domain. Each table uses a composite primary key of (firm_id, fiscal_year, snapshot_id).

Table 3: Fact tables

Fact Table	Stored Variables
`fact_financial_year`	Revenue, expenses, income statement variables, and balance sheet items
`fact_cashflow_year`	`net_cfo`, `net_cfi`, `capex`
`fact_market_year`	`shares_outstanding`, `eps_basic`, `dividend_cash_paid`
`fact_innovation_year`	`rnd_expenses`, `growth_ratio`, `innovation_product_dummy`, `innovation_process_dummy`
`fact_firm_year_meta`	`employee_count`, `firm_age`
`fact_ownership_year`	Ownership structure and equity-related indicators

4 Version Control

The table `fact_data_snapshot` stores metadata for each dataset version, including the snapshot date, a descriptive version tag (e.g., v1_ocr, v2_manual), and a reference to the data source.

This table is the backbone of reproducibility. Every fact record is linked to a snapshot, enabling analysts to query data as of any historical version.

Manual corrections and overrides are logged in `fact_value_override_log`, which records the original value, the new value, the operator who made the change, the timestamp, and a justification note. This provides a complete audit trail for any deviation from the automated extraction.

5 Constraints and Views

Integrity is enforced through the following constraints:

- Unique constraint on ticker in `dim_firm`.
- Composite primary keys on (firm_id, fiscal_year, snapshot_id) in all fact tables.
- Foreign key relationships from all fact tables back to `dim_firm` and `fact_data_snapshot`.

A database view named `vw_firm_panel_latest` returns the most recent snapshot for each firm-year observation, providing analysts with a single, clean query target for the final dataset.

V. Pipeline Design

1 import_firm.py

This script is a sequential **data ingestion (ETL) tool** designed to populate **dimensional tables (DIM tables)** in a MySQL database (vn_firm_panel_test).

The script follows a strict relational order to maintain referential integrity (ensuring foreign keys are created before they are used):

Step 1: Data Sources (dim_data_source)
It iterates through the `source_info` sheet, checking if each data provider already exists in the database. If not, it inserts the source name, type, provider, and notes.

Step 2: Stock Exchanges (dim_exchange)
It extracts unique exchange codes (e.g., HOSE, HNX) from the company list. It verifies their existence in the database and inserts any new stock exchanges.

Step 3: Industry Classifications (dim_industry_l2)
Similar to the exchanges, it scans for unique industry names and populates the industry dimension table, ignoring duplicates.

Step 4: Company Profiles (dim_firm)
This is the core operation. For every row in `company_info`, it checks if the company's `ticker` already exists.

1. If the ticker is new, the script queries the database to grab the newly generated `exchange_id` and `industry_l2_id` (foreign keys) based on the text values in the Excel sheet.
2. It then inserts the full company profile, including the ticker, name, founded year, listed year, and status, successfully mapping the company to its respective exchange and industry.

2 create_snapshot.py

This script acts as the crucial version control mechanism in a larger data pipeline by reading versioning configurations from an Excel file (`version_info` sheet) using `pandas` and generating metadata snapshots in a MySQL database via `pymysql`.

For each configuration row, it cleans missing values and date formats, queries the `dim_data_source` table to resolve the `source_name` into a foreign key (`source_id`), and safely inserts a new tracking record into the `fact_data_snapshot` table.

This entire process is wrapped in a robust transaction block utilizing `commit` and `rollback` to ensure database integrity. Ultimately, the script outputs a list of newly generated `snapshot_ids`, which serve as essential tracking tags for downstream scripts (like `import_panel.py`) to accurately link massive batches of financial metrics to their correct fiscal year, data provider, and version.

3 import_panel.py

This script serves as the primary ETL (Extract, Transform, Load) engine of the project, responsible for orchestrating the flow of complex firm-year data into the structured MySQL environment. It transforms the aggregated "Master Dataset" containing 38 variables into a normalized format across multiple relational tables.

- **Data Sanitization & Normalization:**
  - Handles regional formatting inconsistencies, such as swapping comma and period decimal delimiters found in Vietnamese financial reports.
  - Standardizes missing values (e.g., "n/a", "-", "thieu") into `NULL` types to prevent calculation errors during analysis.

- **Referential Integrity Validation:**
  - Performs a real-time lookup against the `dim_firm` table to maintain strict consistency.
  - Automatically rejects any record with a Ticker not previously registered via `import_firm.py` to prevent "orphan" data.
- **Fact Table Distribution:**
  - Deconstructs the 38 input variables into six specialized FACT tables: `ownership`, `financial`, `cashflow`, `market`, `innovation`, and `firm_year_meta`.
  - This architecture optimizes query performance and allows for modular updates to specific data categories.
- **Snapshot-Based Versioning:**
  - Every entry is tagged with a mandatory `snapshot_id` retrieved from `fact_data_snapshot`.
  - Facilitates the storage of multiple versions for the same fiscal year (2020–2024), ensuring an audit trail for all updates.
- **Atomic Upsert Mechanism:**
  - Employs an "Upsert" strategy based on a composite primary key (Firm-Year-Snapshot).
  - Ensures the pipeline is fully reproducible, allowing for re-runs without creating duplicate records.

4 qc_checks.py

`qc_checks.py` is the quality-control script in the pipeline. It reads data from `vw_firm_panel_latest`, joins several additional fields required for validation, including `founded_year`, latest `share_price`, and latest `evidence_note`, and then applies a set of rule-based checks across firm-year observations. All flagged records are exported to `qc_report.csv` for later manual review and correction.

The script does not directly modify the database. Its main function is to detect suspicious values, logical inconsistencies, accounting mismatches, and missing supporting documentation, then store them in a structured QC report for downstream fixing.

Table 4: QC rules implemented in `qc_checks.py`

Rule group	Variables / fields	QC logic	Error type
Firm age logic	`firm_age`, `founded_year`	`firm_age` must equal `fiscal_year - founded_year`	`AGE_LOGIC_ERROR`
Age progression	`firm_age`, `fiscal_year`	If fiscal years are consecutive, `firm_age` must increase by 1	`PROGRESSION_ERROR`
Time continuity	`fiscal_year`	Detect missing years within a firm time series	`TIME_GAP`
Ownership ratio range	`managerial_inside_own`, `state_own`, `institutional_own`, `foreign_own`	Values must lie within [0,1]	`RANGE_ERROR`

Rule group	Variables / fields	QC logic	Error type
Shares outstanding validity	shares_outstanding	Must be strictly greater than 0	INVALID_VALUE
Non-negative fields	net_sales, total_assets, intangible_assets_net, total_liabilities, cash_and_equivalents, long_term_debt, current_assets, current_liabilities, inventory, net_ppe, wip_goods_purchase, merchandise_purchase_year, firm_age, employees_count	Values cannot be negative	NEGATIVE_VALUE
Non-positive expense fields	selling_expenses, general_admin_expenses, manufacturing_overhead, raw_material_consumption, outside_manufacturing _expenses, production_cost, rnd_expenses, capex, dividend_cash_paid	Values cannot be positive	POSITIVE_VALUE
Growth outlier	growth_ratio	Must remain within the preset bounds $(-0.95, 5.0)$	OUTLIER
Market cap consistency	market_value_equity, shares_outstanding, share_price	Compare reported market value equity with shares_outstanding $\times$ share_price, allowing 5% tolerance	INCONSISTENT
Accounting balance check	total_assets, total_liabilities, total_equity	total_assets should equal total_liabilities + total_equity within 5% tolerance	ACCOUNTING_IMBALANCE
Asset component reasonableness	current_assets, total_assets	current_assets cannot exceed total_assets	COMPONENT_ERROR
Asset composition check	cash_and_equivalents, inventory, current_assets	cash_and_equivalents + inventory cannot exceed current_assets	COMPONENT_ERROR
Fixed/intangible asset check	net_ppe, intangible_assets_net, total_assets	net_ppe + intangible_assets_net cannot exceed total_assets	COMPONENT_ERROR
Liability component check	long_term_debt, current_liabilities, total_liabilities	long_term_debt + current_liabilities cannot exceed total_liabilities	COMPONENT_ERROR
Profit vs. revenue check	net_income, net_sales	net_income cannot exceed net_sales	COMPONENT_ERROR
Innovation dummy completeness	product_innovation, process_innovation	Values cannot be null	MISSING_VALUE
Innovation dummy validity	product_innovation, process_innovation	Values must be binary (0 or 1)	INVALID_DUMMY

Rule group	Variables / fields	QC logic	Error type
Product innovation note consistency	`product_innovation`, `evidence_note`	If `product_innovation` = 1, a `product:` note must exist; if = 0, such a note should not exist	`MISSING_PRODUCT_NOTE`, `UNEXPECTED_PRODUCT_NOTE`
Process innovation note consistency	`process_innovation`, `evidence_note`	If `process_innovation` = 1, a `process:` note must exist; if = 0, such a note should not exist	`MISSING_PROCESS_NOTE`, `UNEXPECTED_PROCESS_NOTE`

The QC rules in `qc_checks.py` can be grouped into four main categories: temporal consistency checks, range and sign validation, accounting consistency checks, and innovation-documentation consistency checks. Together, these rules help identify errors in firm-year continuity, abnormal financial values, balance-sheet inconsistencies, and mismatches between innovation indicators and their supporting notes before the data proceeds to the correction stage.

5 quick_fix.py

`quick_fix.py` is designed as a manual correction tool. Its workflow can be summarized in three steps:

1. A flagged record is reviewed in `qc_report.csv`.
2. The reviewer manually enters the corrected value into the `new_value` field.
3. `quick_fix.py` reads that approved value and updates the latest snapshot record in the database.

If `new_value` is blank, the row is skipped. The script also skips rows with missing `table_name`, ambiguous `column_name`, or invalid `fiscal_year`. After a valid correction is applied, the same value is synchronized back to the Excel master file so that the spreadsheet and database remain consistent.

An important feature of `quick_fix.py` is the audit trail stored in `fact_value_override_log`. Each successful manual override is recorded with the fields `firm_id`, `fiscal_year`, `table_name`, `column_name`, `old_value`, `new_value`, `reason`, `changed_by`, and `changed_at`. These fields document what value was changed, where it was changed, why it was changed, who made the correction, and when the override occurred. As a result, the log table provides a clear record of all manual adjustments applied to the panel dataset.

override_id	firm_id	fiscal_year	table_name	column_name	old_value	new_value	reason	changed_by	changed_at
1	21	2020	fact_innovation_year	evidence_note	Product: nan | Process: https://finance.vietstoc...	Product: Process: https://finance.vietstock.vn/...	Đổi mới SP (1) nhưng ghi chú "product:" trống h...	Group_Member	2026-03-10 07:54:05
2	21	2023	fact_financial_year	net_sales	3.00	2680350000000.0	Lợi nhuận ròng vượt quá doanh thu	Group_Member	2026-03-10 07:54:05
3	21	2022	fact_financial_year	current_assets	1305795538303.00	123433000.0	Tài sản ngắn hạn vượt quá tổng tài sản	Group_Member	2026-03-10 08:03:04

Figure 4: Example of fact value overrite log

6 export_panel.py

`export_panel.py` is the final output step of the pipeline. Its purpose is to extract the cleaned panel data from the database and export it into a CSV file for final use. The script reads data directly from the

view vw_firm_panel_latest, which represents the latest validated version of the panel, and excludes the dummy ticker TEST.

Before exporting, the script sorts the dataset by ticker and fiscal_year so that the final output is organized in a clear firm-year structure. It then writes the result to panel_latest.csv using UTF-8-SIG encoding, which helps preserve compatibility when the file is opened in spreadsheet software. After the export is completed, the database connection is closed by disposing the engine.

In short, export_panel.py serves as the delivery stage of the workflow. It does not transform or validate the data further, but simply extracts the latest cleaned panel and saves it as the final CSV dataset for downstream analysis and reporting.

VI. Outputs

This project produces a set of final outputs that support both analytical use and data-governance requirements. The main deliverable is panel_latest.csv, which is exported from the database view vw_firm_panel_latest. This file represents the latest validated version of the Vietnamese firm financial panel and is structured in firm-year format for downstream analysis and reporting.

In addition to the final panel dataset, the workflow also produces supporting outputs for quality control and auditability. The file qc_report.csv stores all records flagged by the QC layer for manual review and correction. After approved fixes are entered, the applied overrides are recorded in fact_value_override_log, which preserves the history of manual adjustments and their justification.

In coverage terms, the dataset includes 20 Vietnamese listed firms observed over the 2020–2024 period. The panel is built from a master dataset containing 38 variables, which are distributed across specialized fact tables and then consolidated into the final export layer.

Table 5: Main outputs of the pipeline

Outputs	Description
panel_latest.csv	Final cleaned panel dataset exported for analysis and reporting
qc_report.csv	QC report listing flagged firm-year records for manual review

VII. Conclusion

This report presents a complete data engineering workflow for building a Vietnamese firm financial panel dataset for the 2020–2024 period. The project is designed as an end-to-end pipeline that transforms scattered and partially unstructured corporate disclosures into a structured, consistent, and analysis-ready database. Rather than focusing only on extraction, the system covers the full process of data acquisition, AI-based document extraction, manual review, database loading, validation, correction, and final export.

A key strength of the project is the integration of automation and human supervision. The Airflow-based workflow, together with Gemini-supported extraction, improves the speed and scalability of data collection, while Google Sheets review and manual completion help address missing or uncertain values. On the storage side, the DIM–FACT–SNAPSHOT architecture supports referential integrity, ver-

sion control, and flexible querying across firm-year observations. In addition, the QC layer and manual override mechanism improve transparency by ensuring that data issues can be identified, corrected, and logged in a traceable way.

The final outcome of the workflow is a cleaned panel dataset exported from `vw_firm_panel_latest` into `panel_latest.csv`, supported by complementary outputs such as `qc_report.csv`

and `fact_value_override_log`. Together, these components make the system not only useful for downstream analysis and reporting, but also reliable from a data-governance perspective.

Overall, the project provides a practical and reproducible framework for managing firm-level financial data in Vietnam. In future development, the system can be extended by increasing firm coverage, improving extraction performance for more complex report formats, and adding richer monitoring of completeness and data quality. These improvements would make the platform more scalable and more valuable for long-term research and analytical applications.